GPT-6 Luna migration verification

13 October 2026 | Deep Work | Windows 11 | Europe/Berlin

Author: Libor Burian. Evidence qualifications revised 22 October 2026; the underlying experiment and its raw artifacts have not been rerun or rewritten.

Repository: BurnyCoder/deep-work-jarvis-waifu-supervisor. Baseline: `a78a575` on `master`; implementation branch: `feat/gpt-6-luna`.

Question and hypothesis

Can the three existing GPT-5.6 Luna workloads migrate to `gpt-6-luna` while retaining their Responses API contracts, reasoning settings, and image detail? The hypothesis was that changing the model identifiers would suffice. OpenAI's current model page documents image input, structured outputs, and both `medium` and `xhigh` reasoning. Its image-sizing table did not explicitly list GPT-6 Luna, so successful live requests were required rather than assuming exact preprocessing behavior from another model's documentation.

Method and design decisions

Change the three configuration defaults and corresponding environment example; update the same three model variables in the ignored local `.env`. Preserve repository `medium` defaults, local `xhigh` overrides, prompts, output schemas, `original` productivity detail, `low` agent detail, and `gpt-4o-mini-tts` with the existing voice. Retain OpenAI Python 2.44.0 and the existing lockfile: the installed client accepts string model IDs and already supports the request shapes. Reuse production collaborators and existing tests instead of adding a second API implementation or a new runtime abstraction.

This is one compatibility experiment with automated and live phases. No controlled old/new accuracy, cost, or latency comparison was performed. The migration's repository history, official documentation, source, tests, local logs, saved responses, and capture inspection supplied the evidence below.

Automated results

The original experiment account described a test-first red result of one failure and six passes. That red-run transcript is not among the three retained test logs listed below, so its exact execution order and counts were not independently re-established in the 22 October check. The retained green-run logs show that the focused configuration, main wiring, analyzer, and feedback suite passed all 44 tests in 8.27 seconds. The full suite passed all 251 tests in 5.98 seconds. The original account also records a passing `uv lock --check`; its command output was not retained with those test logs. The migration Git diff contains no dependency change. The first focused command named nonexistent `test_messages.py`; file discovery identified `test_feedback.py`, and the corrected command succeeded. This was a test-invocation correction, not a product defect.

Live results

Smoke: the log records that the smoke cycle reached its completion message; the shell exit code is not recorded in that log. Two monitor captures and a successful webcam frame formed the stitched image. The single-capture request returned a valid productivity verdict with HTTP 200, `model=gpt-6-luna`, `reasoning.effort=xhigh`, and `detail=original`. Its first observation used the required no-chronological-comparison wording. Speech was queued, but no Speech API response arrived within smoke's 60-second wait.

Rolling comparison: the saved log records two direct monitor ticks after the smoke completion message. The retained log does not prove the exact shell snippet or the first process's exit time. The two ticks in one unchanged session returned accepted verdicts. The second request contained the two saved captures in chronological order, the `COMPARISON (2/5)` header, and `original` detail for both images. Both inspected composites were 960 by 1896 pixels with the expected monitor and webcam labels. The first longer speech check received HTTP 200 about 114 seconds after queueing; the second Speech API response arrived in about 0.65 seconds. There were no ERROR records in either run log. The original account stated that playback

returned and the queues drained, but the retained logs contain no explicit backend-return or queue-drain evidence, and the exact executed snippet was not retained. Those playback claims were not independently re-established. Audible quality was not measured.

Agent-watch: the existing checker accepted the latest capture with `low` detail and returned a valid `AgentActivityVerdict` through the same new model. Text: the existing message generator returned nonempty good-luck feedback through `gpt-6-luna`. All five successful Responses exchanges (smoke, two rolling captures, watcher, text) persisted full response objects with status `completed`, matching request/response model and `xhigh` reasoning, and no response error. Stored vision requests retained capture paths and detail values without base64 image duplication.

Findings and limitations

The API/schema compatibility hypothesis passed for all three workloads, including both image-detail settings. This does not establish a general image sizing guarantee or an accuracy improvement. Manual inspection found an active Codex elapsed-time status in the image, while the low-detail watcher reported idle. That missed activity is a concrete accuracy limitation; the requested low-detail setting and watcher prompt were preserved. The rolling verdict's reported changed-file count matched the visible captures, but this small, uncontrolled sample is not a productivity benchmark.

The later rolling run's first speech response took about 114 seconds. That is a separate request: it does not establish why the earlier smoke speech did not finish within its wait. The documented verification snippet uses a bounded 180-second wait and checks unfinished queue work; production speech behavior is unchanged. Direct monitor ticks credit the configured five-minute intervals even when invoked sooner, so the reported ten-minute streak is test accounting, not measured elapsed work. No enforcer loop, app-kill sweep, real hosts modification, browser flow, or normal five-minute cadence was exercised in this experiment.

Reproduction and evidence

Use the model-migration checklist for commands, side effects, and acceptance criteria. Start from the repository root with its uv-managed environment. Keep other app instances stopped. Results depend on account access, devices, network, current model behavior, and the visible desktop.

Local evidence is intentionally untracked: `logs/deepwork_20261013_203822.log` and `logs/deepwork_20261013_204040.log`; Responses exchanges are timestamped `20261013_203841_458136`, `20261013_204052_305855`, `20261013_204305_864634`, `20261013_204329_261592`, and `20261013_204332_304992` under `results/llm/`. The first three are vision, followed by agent-watch and message. Capture pixels, observations, session context, and credentials are excluded from this public report. Retained test logs are `logs/luna_focused_20261013_203731.log`, `logs/luna_focused_20261013_203759.log`, and `logs/luna_full_20261013_203822.log`. The PDF is rendered from this Markdown; see the report index for the reproducible renderer command.

Git initially treated the PDF as text. A scoped `reports/*.pdf binary` attribute prevents newline rewriting; the staged blob is verified byte-for-byte against the reviewed artifact (Git attributes).

Review and conclusion

One bounded review covered the changed defaults, independent overrides, request compatibility, privacy, test coverage, maintainability, logging, and documented claims. No new runtime path, schema, dependency, or prompt was introduced. Credential and runtime directories remain ignored. Documentation distinguishes successful model requests from measured accuracy and speech timing. The model upgrade is compatible with the exercised contracts, subject to the observed watcher accuracy and speech latency limitations.

Sources

OpenAI GPT-6 Luna model reference: model ID, modalities, structured outputs, reasoning levels; rechecked 22 October 2026.

OpenAI migration guidance: preserve supported effective reasoning and existing compatible request contracts.

OpenAI image detail and sizing: model-specific detail support and limits; no Luna-specific sizing guarantee inferred.

OpenAI Python client: existing Responses create/parse and speech streaming interfaces.

ReportLab document and paragraph flow: PDF generation from wrapped paragraphs using SimpleDocTemplate.